

SGLang Omni · 每日 PR 学习日报

报告日期：2026-06-16 | 覆盖窗口：过去 24 小时内合并 (merged) 的 PR | 仓库：[sgl-project/sglang-omni](https://github.com/sgl-project/sglang-omni)

- 多模态 / Omni
- TTS 语音合成
- 压力测试 / 基准
- CI 基础设施

当日概览

过去 24 小时 (2026-06-15 ~ 2026-06-16) 共有 8 个 PR 被合并。按主题可以分成三类：

8

合并 PR 总数

3

架构 / 功能类 (深讲)

5

CI / 校准类 (速览)

5

不同贡献者

主题分类

类别	PR	一句话
多模态压测	#770	给 Qwen3-Omni 写一个并发/多 rollout 压力测试脚本
TTS 新功能	#662	给语音合成服务加"上传自定义音色"的一整套 API
多模型 CI	#727	让 TTS CI 支持多模型随机抽测 (新增 MOSS)
CI 校准/基建	#796 #795 #794 #793 #806	阈值重新标定、CI 主机画像、预检清单、权限

本期重点讲透的 3 个 PR

#770 · [\[Qwen3-Omni\] benchmarks: concurrency / rollout stress runner](#) — 并发压测、异步 IO、p50/p95/p99、RTF

#662 · [TTS] Add uploaded voice APIs — 多阶段流水线、说话人缓存、声纹/参考音频、LRU

#727 · feat(ci): add MOSS TTS to TTS CI with random model selection — CI 设计、阈值门控、可观测但不阻断

本报告遵循"讲深 > 讲全"原则：前 3 个与 Omni / 多模态 / 多阶段架构强相关的 PR 逐块拆解代码与概念；后 5 个 CI 类放在末尾统一速览。

#770 · [Qwen3-Omni] benchmarks : 并发 / rollout 压力测试器

作者 [luojiaxuan](#)

[merged](#)

· +333 行 / 1 文件

· [查看 PR](#) ↗

1. PR 概览

- 标题 : [Qwen3-Omni] benchmarks: concurrency / rollout stress runner
- 编号 / 作者 / 状态 : #770 / [luojiaxuan](#) / 已合并 (由 [yxs](#) 合并)
- 改了什么 : 新增一个独立的压测脚本 (合并后落在 [benchmarks/eval/benchmark_omni_rollout_stress.py](#)) , 用同一个 prompt 在一串递增的并发档位 (1,2,4,8,16,32...) 下反复打服务器, 记录每档的 req/s、输出 token/s、prompt token 数、以及 p50/p95/p99 延迟。不改任何运行时代码。

一句话

这是一把“压力测试卷尺”: 它不改模型、不改引擎, 只是反复地、并发地向已经跑起来的 Qwen3-Omni 服务发请求, 量出“并发越高, 服务还撑不撑得住、快不快”。

2. 它解决了什么问题

在做推理服务时, 有一类非常典型的流量叫“同一个 prompt 同时跑很多份” (multi-rollout) ——比如强化学习采样、Best-of-N 采样、A/B 一致性测试, 都会出现“一模一样的输入并发 N 份”。这种流量对 KV cache 复用、batch 调度的压力跟普通流量不一样。

此前仓库里缺一个标准工具去回答这些问题: “并发拉到 32 的时候吞吐还能线性增长吗? 尾延迟 (p99) 会不会爆? 服务会不会 OOM 或挂掉?” (对应 issue #760 要排查的并发服务特性)。没有这把尺子, 优化前后就没有可比的数字。这个 PR 就是把这把尺子做出来。

为什么需要

“没有测量就没有优化。”要证明一个性能改动 (比如 #771 的 prefill batching) 真的有效, 你得先有一个能稳定复现、能出数字的压测脚本, 跑出 before/after 两组数对比。#770 就是这组对照实验的“测量仪器”。

3. 具体做了什么改动

只新增一个文件, 全部是异步并发的 HTTP 压测逻辑。核心由几块拼成: 构造请求 → 并发执行 → 统计指标 → 落盘 JSON。下面逐块讲。

3.1 把一个样本“复制”成 N 份

```
# 把同一个样本复制 count 份, 只改 sample_id 让每份可区分
def _duplicate_prompt(sample: MMMUSample, count: int) ->
    list[MMMUSample]:
    return [
        replace(
```

```

        sample,
        sample_id=f"{sample.sample_id}:rollout-n{count}-{index}",
    )
    for index in range(count)
]

```

这里 `replace` 来自 `dataclasses`，作用是“复制一个 dataclass，并只改其中一个字段”。它把同一个多模态样本复制成 `count` 份，内容完全一样，只有 `sample_id` 不同（加了 `rollout-nN-i` 后缀），这样统计时能把每份请求区分开。这正是“same-prompt multi-rollout”的本质——输入相同，只是并发的份数不同。

3.2 单条请求怎么发 (send_fn 工厂)

```

async def send_fn(session, sample) -> RequestResult:
    payload = {
        "model": model_name,
        "messages": [{"role": "user", "content": sample.prompt}],
        "images": [image_to_data_uri(img) for img in sample.images],
        "modalities": modalities,          # ["text", "audio"] 或
["text"]
        "max_tokens": max_tokens,
        "stream": False,
        "user": rollout_group_id,          # 同一组 rollout 用同一个 user
    }
    标记
    if enable_audio:
        payload["audio"] = {"format": "wav"}

    start_time = time.perf_counter()
    async with session.post(api_url, json=payload) as response:
        body = await response.json()
        if response.status >= 400:
            result.error = body.get("detail", str(body))
            return result
        ...
        if enable_audio:
            audio_b64 = (message.get("audio") or {}).get("data", "")
            base64.b64decode(audio_b64)    # 解码一次，确保音频没在传输中损坏
            ...
            usage = body.get("usage", {})
            result.prompt_tokens = int(usage.get("prompt_tokens", 0))
            result.completion_tokens = int(usage.get("completion_tokens", 0))
            result.engine_time_s = time.perf_counter() - start_time
            if result.completion_tokens > 0 and result.engine_time_s > 0:

```

```
result.tok_per_s = result.completion_tokens /
result.engine_time_s
```

逐点拆解：

- 它是个“工厂函数”：`_make_rollout_send_fn(...)` 把模型名、URL、是否要音频等参数固定下来，再返回内层的 `send_fn`。这样压测主循环只管“调用 `send_fn`”，不用关心一堆参数——这是闭包/工厂模式的典型用法。
- 多模态请求：payload 同时带了文本 (`messages`)、图片 (`images`，编码成 data URI)、以及可选的音频输出 (`modalities` 含 "audio")。这正是 Omni 模型的特点——一次请求里同时有多种模态。
- 计时用 `perf_counter()`：它是单调高精度时钟，专门用来测时间间隔（不会被系统改时间影响），比 `time.time()` 更适合测延迟。
- 音频只解码不落盘：`base64.b64decode` 解一次，目的只是“验证音频数据完整、没在 base64 传输里坏掉”，但不为每条请求写 WAV 文件——避免磁盘 IO 干扰压测纯度。这是个很克制的工程细节。
- 从 `usage` 字段读 `token` 数，算出 `tok_per_s`（每条请求的输出速度）。

3.3 并发扫描 + 收集指标（主循环）

```
for count in args.rollout_counts:          # 例如 [1,2,4,8,16,32]
    duplicated = _duplicate_prompt(base_sample, count)
    runner = BenchmarkRunner(
        RunConfig(
            max_concurrency=count,          # 并发度 = 当前档位
            request_rate=float("inf"),      # 不限速：能多快发就多快发（闭环
压测)
            warmup=0,
            timeout_s=args.timeout_s,
        )
    )
    request_results = await runner.run(duplicated, send_fn)
    speed = compute_speed_metrics(request_results,
wall_clock_s=runner.wall_clock_s)
    failed = [r for r in request_results if not r.is_success]
    print_speed_summary(speed, args.model, count, title=f"...
N={count}")
```

这就是压测的“扫频”过程：把并发度从 1 一路加到 32，每个档位跑一批，算一组速度指标。

`request_rate=inf` 表示“闭环压测”——不人为限速，一个请求回来立刻补下一个，让系统始终被 `max_concurrency` 条请求“顶满”，从而测出系统在该并发下的真实上限。

3.4 服务端事件 profiler + 健康检查

跑之前先打 `/health`，并（默认）调用 `/start_request_profile` 让服务器端记录细粒度事件；跑完调用 `/stop_request_profile`，再打一次 `/health`。注意 `finally` 块保证“无论中途是否报错，profiler 一定会被关掉”——这是资源清理的好习惯。最后把 config、profile 信息、前后 health、各档结果一起写进 `rollout_stress_results.json`。

4. 涉及的知识点（小白展开）

并发（concurrency）vs 并行（parallelism）

是什么：并发是“同时有很多任务在进行中”（哪怕 CPU 在它们之间快速切换）；并行是“真的有多核同时算”。

为什么需要：压测要模拟“很多用户同时请求”，关注的是服务能不能同时受理很多请求，而不是本机算得多快。

本 PR 怎么用：用 Python 的 `asyncio` + `aiohttp` 实现“单线程高并发 IO”——程序大部分时间在等服务器回包，等的时候就切去发别的请求，所以一个进程就能压出几十路并发。

async / await 与异步 IO

是什么：一种写法，让“等待网络返回”这种空档不浪费——遇到 `await` 就把 CPU 让给别的协程。

类比：一个服务员（单线程）同时招呼很多桌：A 桌点完菜（发出请求）他不傻等上菜，转身去 B 桌点单，菜好了再回来端（响应回来）。

本 PR 怎么用：`send_fn` 是 `async` 函数，`BenchmarkRunner` 用 `asyncio.run` 同时跑 `count` 个 `send_fn`，实现高并发发包。

p50 / p95 / p99 尾延迟（tail latency）

是什么：把所有请求的延迟排序，p50 是中位数（一半请求比它快），p95/p99 是“最慢的 5%/1% 的门槛”。

为什么需要：平均值会骗人——平均 100ms 看着不错，但 p99 可能是 5 秒，意味着 1% 的用户体验极差。线上 SLA 通常盯 p95/p99。

本 PR 怎么用：`compute_speed_metrics` 对每个并发档算出这些分位数，方便看“并发拉高后，尾延迟是否恶化”。

RTF（Real-Time Factor，实时率）

是什么：生成 1 秒音频需要花多少计算时间。RTF=0.23 表示生成 1 秒音频只用 0.23 秒，即 ~4 倍实时速度。

为什么需要：语音/Omni 的输出是音频，吞吐不能只看 token/s，还要看“能不能比实时更快地产出音频”。RTF<1 才能流式实时播放。

本 PR 怎么用：压测脚本对启用音频输出的请求统计速度，RTF 是衡量 TTS/talker 阶段是否够快的关键指标（也贯穿后面 #796 的校准报告）。

闭环（closed-loop）压测 vs 开环（open-loop）

是什么：闭环=固定并发数，一个回来才发下一个（系统永远只有 N 个在飞）；开环=固定到达速率，不管系统处理得过来与否一直发。

为什么需要：闭环能干净地测“系统在 N 并发下的稳态能力”；开环能测“过载时会不会雪崩”。

本 PR 怎么用：`request_rate=float("inf")` + `max_concurrency=count` 就是经典闭环：用并发度而非速率来施压。

为什么放进 `benchmarks/` 而不是 `eval/`

背景：PR 描述强调它是"serving 压力/诊断工具"，不是"模型能力评测"。作者刻意不想让它被当成标准精度指标。

启示：目录结构也是一种"语义契约"——把诊断工具和能力评测分开放，能避免别人误用你的脚本当成 KPI。（注：最终合并版落在 `benchmarks/eval/` 下，文件名 `benchmark_omni_rollout_stress.py`，与描述里早期路径略有出入，说明评审中调整过位置。）

5. 我能学到什么 / 延伸思考

- 可复用的工程思想：①"先做尺子再做优化"——任何性能工作都应先有可复现的测量工具；②工厂函数 + 闭包来固定配置、简化主循环；③ `finally` 保证 profiler/资源一定被释放；④克制的副作用（音频只解码不落盘）保证测量纯度。
- 想深入该补的前置知识：Python `asyncio` 协程模型、`aiohttp` 客户端；分位数/尾延迟与排队论（Little's Law：并发 $\approx$ 吞吐 $\times$ 延迟）；推理服务的 continuous batching（见 #662 概念区）；以及 RL rollout / Best-of-N 采样为什么会产生"同 prompt 高并发"流量。

#662 · [TTS] 上传自定义音色 (uploaded voice) API

作者 Ratish1 · merged · +2280 / -44 行 · 27 文件 · 查看 PR ↗

1. PR 概览

- 标题：[TTS] Add uploaded voice APIs
- 编号 / 作者 / 状态：#662 / Ratish1 / 已合并 (由 zhaochenyang20 合并)
- 改了什么：给 OpenAI 兼容的 TTS 服务加了一整套"上传并复用自定义音色"的能力。用户可以：`POST /v1/audio/voices` 上传一段参考音频注册一个音色 → 在 `POST /v1/audio/speech` 里用 `voice="我的音色"` 复用它 → `GET` 列出 → `DELETE` 删除。

一句话

以前你想用自己的声音让模型说话，得每次请求都把参考音频塞进去；现在你"上传一次、起个名字"，之后只要报名字就行，而且系统会把这个音色对应的"声纹特征"缓存起来，重复使用时不再重算。

2. 它解决了什么问题

TTS (文字转语音) 里有个高级玩法叫"voice cloning / 参考音频克隆"：你给一段某人的录音，模型就能用那个人的音色把任意文字念出来。此前在这个仓库里，要用自定义音色，每次请求都得把参考音频 (甚至预处理出来的声纹) 一起传过来。痛点有三：

- 重复传输：同一个音色用 100 次，参考音频就得传 100 次，浪费带宽。
- 重复计算：每次都要从参考音频里重新算"声纹特征 / 参考编码"，慢且费 GPU。
- 各模型各写一套：Higgs、Qwen3-TTS 各有各的参考音频格式，调用方得知道用的是哪个模型、怎么塞——耦合严重。

为什么需要

把"音色"从"一次性请求参数"升级成"有生命周期的、可命名的、被服务持久化管理的资源" (上传/列出/删除)，同时在内部做缓存复用。这既对齐了 OpenAI 的 `/v1/audio/voices` 习惯用法，又把"算一次声纹、用很多次"的性能红利拿到手。

3. 具体做了什么改动

这是个跨 27 个文件的较大 PR，但骨架很清晰，核心是新增两个文件 + 在多模型流水线上"接线"：

文件	角色
<code>serve/speech_voices.py</code> (新, 556 行)	音色仓库：上传校验、持久化为 .safetensors、列出/删除、restore-on-startup
	进程级共享的声纹"特征产物" LRU 缓存

<code>scheduling/speaker_cache.py</code> (新, 129 行)	
<code>serve/openai_api.py</code> (+178)	挂上 GET/POST/DELETE 三个路由
<code>serve/speech_service.py</code> (+97)	请求时把"音色名"翻译成模型要的参考音频
<code>models/higgs_tts/*</code> , <code>models/qwen3_tts/*</code>	把上传音色接进各自的参考音频 / x-vector 路径与缓存

3.1 架构：分层 + "下沉" (lowering)

PR 的关键设计是一句话："上传音色由 HTTP 服务层拥有，在请求到达模型流水线之前，被翻译 (lower) 成各模型已有的参考音频路径。" 用图表示：



"lowering (下沉/降级翻译)"是编译器借来的词：高层抽象 ("音色名 my_voice") 在进入底层执行 (模型流水线) 前，被翻译成底层早就懂的东西 ("一段参考音频 ref_audio")。好处是模型层完全不用知道"上传音色"这个概念——它只看到自己熟悉的参考音频。`SpeechService` 因此不需要去解析模型名、不需要给每个 TTS 家族写 if-else。是否需要把 voice 名解析成上传音色，由各模型的 config 用一个能力开关声明。

设计模式

这是"能力声明 (capability flag) + 适配器"：与其在通用代码里写 `if model == "higgs": ...`，不如让每个模型 config 自己声明"我需不需要把命名 voice 解析成上传音色"。新增模型时不用改通用逻辑——这就是"对扩展开放、对修改封闭" (开闭原则)。

3.2 持久化：为什么用 .safetensors

每个上传音色被存成一个 `.safetensors` 文件 (看 `UploadedVoice.to_safetensors_metadata()`，把 name/consent/created_at/mime_type/采样率/采样点数/指纹等写进文件元数据)。重启时 `restore-on-startup` 会把它们重新加载回注册表。还做了上传校验：

```
# speech_voices.py 里的若干安全/校验常量
MAX_VOICE_UPLOAD_BYTES = 10 * 1024 * 1024 # 上传上限 10MB, 防止打爆磁盘/内存
MIN_REFERENCE_AUDIO_SECONDS = 1.0 # 参考音频太短克隆不出音色
MAX_REFERENCE_AUDIO_SECONDS = 30.0 # 太长没必要、还拖慢
VOICE_NAME_PATTERN = re.compile(r"^(?=.*[A-Za-z0-9])[A-Za-z0-9_.-]+") # 音色名白名单, 挡路径穿越
```

```
ACCEPTED_VOICE_UPLOAD_MIME_TYPES = frozenset({"audio/wav", "audio/mpeg", ...})
```

逐点看意图：上传是外部输入，每一条都是潜在攻击面。限制大小（防 DoS）、限制时长（业务约束）、用正则白名单约束音色名（防止 `../../../../etc/passwd` 这种路径穿越）、限制 MIME 类型（只收音频）。这些是写“接受用户上传”的接口时的标准防御动作。

3.3 共享声纹缓存 (speaker_cache.py) ——本 PR 最值得学的一块

```
class SpeakerArtifactCache:
    """进程级、有上限的 LRU, 缓存被多个 TTS 阶段共享的声纹特征."""
    def __init__(self, max_bytes=512*1024*1024): # 默认 512MB 上限
        self._cache = StageOutputCache(max_bytes=max_bytes,
            size_fn=estimate_cache_bytes)
        self._lock = RLock() # 多线程安全

    def get(self, key: SpeakerCacheKey):
        with self._lock:
            value = self._cache.get(_encode_key(key))
            if value is None:
                self._miss_count += 1; return None
            self._hit_count += 1; return value

    def clear_voice(self, voice_name): # 删/改音色时, 精准失效它的所有缓存
        normalized = voice_name.lower()
        with self._lock:
            removed = self._cache.remove_if(
                lambda key: _encoded_key_voice_name(key).lower() ==
                normalized)
            self._delete_invalidation_counter += removed
```

这块在干什么、为什么这么写：

- 缓存 key 是个结构化身份：`SpeakerCacheKey(model_type, voice_name, voice_version, artifact_kind)`。注意 `voice_version`——同一个名字的音色被覆盖上传后，version 变了，老缓存自然不会被命中（避免“换了音色却用了旧声纹”的脏数据）。`artifact_kind` 区分不同种类产物（波形 / ref-code / x-vector）。
- 按字节数限容的 LRU：`estimate_cache_bytes` 会针对 numpy 数组、torch 张量（`numel*element_size`）、dict/list 递归地估算占用内存，超过 512MB 就按“最近最少使用”淘汰。为什么按字节而不是按条数？因为声纹张量大小差异很大，按字节才能真正控制住显存/内存。
- 线程安全：用 `RLock` 包住读写，因为服务是多线程/多请求并发访问同一个全局缓存。
- 精准失效：删除或覆盖音色时调 `clear_voice`，只清掉这个音色相关的缓存条目（`remove_if` + 从 key 里反解出 voice_name 比对），不会误伤别的音色。PR 描述里强调“per-voice invalidation 让 Higgs 和 Qwen 的缓存被一致地清掉”——这就是它。

- 全局单例：文件底部 `GLOBAL_SPEAKER_CACHE = SpeakerArtifactCache()` + `get_speaker_artifact_cache()`，保证整个进程共享一份缓存，于是 Higgs 阶段算出的产物、Qwen 阶段也能从同一个缓存受益。

3.4 接进各模型流水线

真正“省算力”的地方在模型层接线：**Higgs** 把上传音色的元数据接进 waveform / ref-code 缓存，重复用同一音色时跳过“可复用的 prompt 产物”重算；**Qwen3-TTS Base** 接进参考音频路径，并新增“**x-vector-only** 模式”——当上传时没有配套转写文本（ref_text）时，只用说话人嵌入向量（x-vector）而不需要 ref_code 张量也能合成。这让“只有一段录音、没有文字稿”的音色也能用。

4. 涉及的知识点（小白展开）

SGLang 的多阶段（multi-stage）流水线架构

是什么：这个仓库副标题就是“High-Performance Multi-Stage Pipeline Framework for Omni Models”。Omni 模型把一次请求拆成多个“阶段”接力处理：例如 thinker（理解输入、生成文本）→ talker（把文本变成语音 token）→ vocoder/code2wav（把 token 变成波形）。

为什么需要：多模态生成天然是流水线——先想内容，再决定怎么发音，再合成声波。分阶段能各自优化、各自缓存、甚至各自用不同并行度。

本 PR 怎么用：声纹特征是被多个阶段共享的中间产物，所以缓存放在 `scheduling/` 下、命名为 `SpeakerArtifactCache`，并复用已有的 `StageOutputCache`（阶段输出缓存）作为底座。

参考音频 / 声纹 / x-vector / speaker embedding

是什么：参考音频=一段示例录音；从中可以抽出“说话人嵌入”（speaker embedding，一种代表说话人身份/音色的定长向量），x-vector 是其中一种经典实现。

为什么需要：模型不能直接“听懂”原始波形里的身份信息，需要先把音色压成一个向量喂给生成器，它才能“用这个人的嗓子”说话。

本 PR 怎么用：把“算一次 embedding/ref-code，缓存起来反复用”作为核心优化；x-vector-only 模式让没有文字稿的录音也能克隆。

LRU 缓存（Least Recently Used）

是什么：一种淘汰策略——内存满了，先扔“最久没被用过”的。

类比：书桌只能放 10 本书，新书要放上来时，把最久没翻的那本先收进书柜。

本 PR 怎么用：`SpeakerArtifactCache` 用按字节限容的 LRU 装声纹产物，512MB 满了就淘汰最旧的，避免无限增长把显存/内存吃光。

缓存失效（cache invalidation）

是什么：当底层数据变了，缓存里的旧值必须作废，否则会读到“脏数据”。计算机科学有句名言“缓存失效是两大难题之一”。

本 PR 怎么用：两条防线——① `voice_version` 进 key，覆盖上传自动让旧 key 失配；②删除/覆盖时 `clear_voice` 精准清除该音色全部产物。两者配合保证“换了音色，绝不用旧声纹”。

.safetensors 文件格式

是什么：Hugging Face 推的张量序列化格式，相比 pickle 更安全（加载时不会执行任意代码）、可零拷贝映射、还能附带一段键值元数据。

本 PR 怎么用：每个上传音色存成一个 .safetensors：张量部分放音频/特征，metadata 部分放 name/consent/采样率/指纹等。重启后能完整还原。

OpenAI 兼容 API / 资源型接口设计 (REST)

是什么：用 `POST` (创建) / `GET` (列出) / `DELETE` (删除) 这套 HTTP 动词把“音色”当成可增删查的资源管理，且路由 (`/v1/audio/voices`, `/v1/audio/speech`) 对齐 OpenAI 习惯。

为什么需要：对齐既有生态，调用方几乎零学习成本就能迁移过来。

本 PR 怎么用：把音色从“一次性参数”提升为“有生命周期的资源”，配合服务端持久化，这正是从“功能”到“产品化能力”的跨越。

5. 我能学到什么 / 延伸思考

- 可复用的工程思想：①抽象下沉 (**lowering**) ——在边界处把高层概念翻译成底层已懂的东西，让底层零改动；②能力开关代替类型判断——让各实现自己声明能力，通用层不写 if-else，新增模型零侵入；③缓存三要素：结构化 key (含 version)、限容淘汰 (按真实字节)、精准失效 (per-key)；④把用户上传当攻击面：大小/时长/名字/MIME 全校验。
- 想深入该补的前置知识：TTS / voice cloning 基础 (mel 频谱、vocoder、speaker embedding)；REST 资源建模；并发下的锁与线程安全；HF safetensors；以及 SGLang 的 stage 缓存机制 (`StageOutputCache`) ——它是本 PR 缓存的底座。

#727 · feat(ci) : 给 TTS CI 加 MOSS 模型 + 随机抽选机制

作者 yijiangt · merged · +1157 / -224 行 · 12 文件 · 查看 PR ↗

1. PR 概览

- 标题 : feat(ci): add MOSS TTS to TTS CI with random model selection (issue #724)
- 编号 / 作者 / 状态 : #727 / yijiangt / 已合并 (由 zhaochenyang20 合并)
- 改了什么 : 把 TTS 的持续集成 (CI) 从"只测 Higgs 一个模型"扩展成"支持多个模型, 每次 CI 随机抽一个跑" (当前是 Higgs 或 MOSS), 从而在不增加单次 CI 成本的前提下扩大覆盖面。

一句话

测两个模型如果每次都全测, CI 时间和 GPU 成本翻倍; 这个 PR 用"每次随机摇一个测"的办法, 保持单次成本不变, 长期看两个模型都会被测到。

2. 它解决了什么问题

CI (持续集成) 是"每次提交代码后自动跑的一批测试"。TTS 的 CI 要在真实 GPU 上把模型起起来、跑完整的 SeedTTS 数据集、测流式和非流式——非常贵、非常慢。现在要新增 MOSS 模型, 如果每次提交都把 Higgs + MOSS 都跑一遍, CI 成本直接翻倍, 开发者等得更久。

为什么需要

在"覆盖更多模型"和"控制 CI 成本"之间找平衡。随机抽选是个聪明的折中: 单次只跑一个 (成本恒定), 但因为是随机的, 几十次提交累计下来每个模型都被反复测到——用"统计意义上的覆盖"换"每次全覆盖"的高成本。

3. 具体做了什么改动

改动落在三层: GitHub Actions 工作流 (怎么调度)、测试代码 (怎么参数化模型)、阈值配置 (每个模型各自的及格线)。

3.1 工作流里加一个"摇号"job

```
# .github/workflows/omni-ci.yaml
pick-tts-model:
  needs: setup
  runs-on: ubuntu-latest
  outputs:
    tts_ci_model: ${ steps.pick.outputs.tts_ci_model }
  steps:
    - id: pick
      env:
        TTS_CI_MODEL_OVERRIDE: $
```

```

{{ github.event.inputs.tts_ci_model }}
run: |
    models=(higgs moss)
    override="${TTS_CI_MODEL_OVERRIDE,,}" # ,, = 转小写
    if [[ -n "${override}" ]]; then
        # 手动指定了就用指定的（且校验必须是合法模型，否则报错退出）
        ...
    else
        selected="${models[$((RANDOM % ${#models[@]}))]}" # 否则
        随机摇一个
    fi
    echo "tts_ci_model=${selected}" >> "$GITHUB_OUTPUT"

```

逐点拆解：

- 新增一个 `pick-tts-model` job，它依赖 `setup`，跑在便宜的 `ubuntu-latest`（不占 GPU）。它的唯一职责就是“决定这次测哪个模型”，把结果通过 `outputs` 传给下游。
- 两种来源：① `workflow_dispatch` 手动触发时可传入 `tts_ci_model` 指定模型（方便复现某次失败）；② 没指定就用 `$RANDOM % 模型数` 随机选。`${VAR,,}` 是 bash 把变量转小写的写法。
- 手动指定会校验合法性：如果传了个不存在的模型名，直接 `::error::` 报错退出——防止手滑跑了个无意义的 CI。
- 结果通过 `$GITHUB_OUTPUT` 传递：下游 `tts-ci` job 用 `with: tts_ci_model: ${{ needs.pick-tts-model.outputs.tts_ci_model }}` 接收，再作为 `TTS_CI_MODEL` 环境变量传进测试的 2/3/4 阶段。

3.2 用 preset 数据类把“模型差异”参数化

```

# tests/test_model/tts_ci_config.py (新文件)
@dataclass(frozen=True)
class TtsCiModelPreset:
    model_path: str
    ref_format: Literal["flat", "references"] = "flat"
    gate_thresholds: bool = True # 关键：这个模型是否“门控”（不达标就让
    CI 失败)
    num_gpus_per_worker: int = 1
    ...

TTS_CI_PRESETS = {
    "higgs": TtsCiPreset(model=..., thresholds=...), # Higgs：有及格
    线，会门控
    "moss": TtsCiPreset(...), # MOSS：
    gate_thresholds=False, 只测量不阻断
}

```

这里把“每个模型的差异”全收进一个 `preset` 字典里：模型路径、参考格式、每 worker 几块 GPU、以及最关键的 `gate_thresholds`。新增一个模型只需 3 步：往 `TTS CI PRESETS` 加一条、往 workflow `models=(...)` 加个名字、往下载脚本加 HF 仓库 ID。这是教科书式的“用数据驱动代替改逻辑”。

3.3 “测量但不门控” (no-gate) 模式

对 MOSS (`gate_thresholds=False`)：所有指标照常测、照常打印，但前缀打成 `[TTS CI] ... (no gate)`，且不会因为不达标而让 CI 失败。为什么这么设计？因为一个新接入的模型还没有可靠的基线/阈值，如果一上来就拿它当硬门槛，CI 会频繁误红。先用“只观测”模式跑一阵，攒够稳定数据（像 #796 那样校准出阈值），再把它升级成门控。这是接入新模型的稳妥姿势。

阈值是怎么来的

`tts_ci_config.py` 里能看到：先有 P95 参考值（如 Higgs 非流式 throughput 14.717 qps），再乘“松弛系数”得到阈值——`THRESHOLD_SLACK_HIGHER=0.75`（越大越好的指标，门槛 = $P95 \times 0.75$ ，留 25% 余量）、`THRESHOLD_SLACK_LOWER=1.25`（越小越好的指标如延迟，门槛 = $P95 \times 1.25$ ）。留余量是为了容忍正常的机器抖动，不让 CI 因为偶发波动就误报。

4. 涉及的知识点（小白展开）

CI / 持续集成 与 GitHub Actions

是什么：CI = 每次提交代码后自动跑测试，尽早发现问题。GitHub Actions 是 GitHub 内置的 CI 系统，用 YAML 描述“哪些 job、什么时候跑、依赖关系”。

为什么需要：人工每次手动测既慢又会忘；自动化保证“每次改动都被同一套标准检验”。

本 PR 怎么用：新增 `pick-tts-model` job 用 `needs` 串起依赖、用 `outputs` 在 job 间传值，是 Actions 编排的标准手法。

随机抽样测试 (sampling-based CI)

是什么：不是每次全测，而是每次随机抽一部分测，靠多次累积达到覆盖。

类比：工厂质检不会拆开每一个产品，而是每批随机抽几个；只要抽得够多够随机，长期就能发现问题。

本 PR 怎么用：`$RANDOM % 模型数` 每次摇一个模型，单次成本恒定，长期两个模型都被覆盖。

门控 (gating) vs 观测 (no-gate)

是什么：门控=指标不达标就让 CI 变红、阻止合并；观测=只测量记录、不阻断。

为什么需要：成熟的、有可靠基线的指标才适合做硬门槛；新接入、基线未知的指标先观测，避免误报淹没真问题。

本 PR 怎么用：Higgs 门控、MOSS 先 no-gate，等校准出稳定阈值再升级。

阈值松弛系数 (slack / 容差) 与 P95 基线

是什么：用历史多次运行的 P95（95 分位）当基线，再乘一个余量系数得到 CI 门槛。

为什么需要：性能数字天然会抖动；门槛卡太死会频繁误红（flaky），太松又抓不到真退化。留余量是工程上的平衡。

本 PR 怎么用：higher-is-better 用 $\times 0.75$ 、lower-is-better 用 $\times 1.25$ 。这套机制和当天的 #796 校准报告是同一套体系。

数据驱动配置（data-driven / 配置即代码）

是什么：把“会变的东西”（模型列表、阈值）抽成数据结构/配置，让“逻辑代码”保持稳定。

本 PR 怎么用： `TtsCiModelPreset` / `TTS_CI_PRESETS` 把模型差异收进 dataclass 字典，新增模型只加数据、不改逻辑——降低出错面、易于评审。

5. 我能学到什么 / 延伸思考

- 可复用的工程思想：①成本/覆盖的折中——随机抽样把“全测的高成本”换成“统计覆盖”；②新东西先观测后门控——降低引入风险；③用便宜的 **job** 做决策（摇号在 CPU runner 上，GPU 只留给真测试）；④数据驱动把可变项与逻辑分离。
- 想深入该补的前置知识：GitHub Actions 的 `needs` / `outputs` 可复用 workflow (`workflow call`)；flaky test 与统计显著性；性能基线校准方法（见 #796）；以及 WER / UTMOS / 说话人相似度这些 TTS 评测指标的含义。

速览：CI / 校准 / 基建类 PR (#796 #795 #794 #793 #806)

主要由 zhaochenyang20 推动 · 不改运行时模型逻辑 · 属于“测试基础设施”层

这 5 个 PR 不涉及模型/推理逻辑本身，但它们是支撑前面那些功能 PR 能被“可信地测量”的地基。简要说明每个在做什么、为什么有价值：

#796 · [CI] Calibration 2026/06/15 (阈值重新标定)

做了什么：在 2×NVIDIA H20 上把 TTS / Qwen3-ASR / 以及一大批 Omni 评测 (MMM、MMSU、VideoMME、VideoAMME, 含 talker 音频与 TP2 变体) 各跑 5 遍，取“5 次中最差” (worst-of-5) 作为新的 CI 阈值。一次性更新了几十个门槛常量。

为什么有价值：硬件、依赖版本 (sglang 0.5.12 / torch 2.11 / CUDA 13) 变化后，旧阈值要么过松 (抓不到退化) 要么过紧 (误红)。用“worst-of-N”标定是个稳健做法——以最差表现为底线，能容忍正常抖动，又不放过真退化。值得注意的是不少 ASR 速度阈值大幅收紧 (如 p95 延迟 0.555s → 0.166s, 紧了 70%)，说明底层性能其实变好了，于是把门槛提高以锁住收益。

可学到

“worst-of-N 校准”是性能回归测试里对抗抖动的经典套路：跑多遍取最差当基线，配合松弛系数，得到既稳又有意义的门槛。这和 #727 的阈值体系是同一套方法论。

#794 · [CI] CI Host Profiles (+556 行) / #795 · Add Pre Check List / #793 · Update Calibration Skills

这三个是 #796 校准能可信进行的“配套基建”：

- **#794 主机画像**：把不同 CI 机器 (H20/H100/H200...) 的环境参数固化成“画像”配置。为什么重要：性能数字只有在注明硬件时才有意义——同一阈值在 H20 和 H200 上完全不可比。主机画像让校准结果可追溯、可复现。
- **#795 预检清单 (Pre Check List)**：跑校准前先核对一份清单 (机器、venv、样本数、并发等是否就绪)，避免“环境没配对就跑出一堆无效数据”。
- **#793 更新校准技能 (skills)**：更新仓库里 `tune-ci-thresholds` 这套“校准工具/技能”的脚本与文档，让校准流程标准化、半自动化。

可学到

“可信的基准测试”本身是一项工程：要有主机画像 (在哪测)、预检清单 (测之前确认环境)、标定脚本 (怎么取阈值)、和 provenance (记录分支/版本/时间)。数字才不会变成“没人敢信的红绿灯”。

#806 · [CI]: Add luojiaxuan to CI permissions (+5 行)

做了什么：往 `.github/CI_PERMISSIONS.json` 里给贡献者 `luojiaxuan` 加 `/tag-run-ci-label` 和 `/rerun-failed-ci` 权限。为什么有价值：GPU CI 很贵，所以只有被授

权的人能用斜杠命令触发/重跑 CI——这是把昂贵资源的触发权限做成显式白名单的访问控制实践，纯权限维护，无逻辑改动。

今日总串讲：这 8 个 PR 串起来看

把今天的 PR 放在一起，能看到一条很完整的“工程化推理服务”叙事线：

- 功能层 (#662)：把“自定义音色”从一次性参数升级成可持久化、可缓存的资源——体现抽象下沉 + 能力开关 + 缓存三要素。
- 测量层 (#770)：写专门的并发压测仪器去量化“系统在高并发下还行不行”——体现先有尺子再优化、闭环压测、尾延迟。
- 验证层 (#727 + #796/795/794/793)：把测试做成可扩展（多模型随机抽）、可信（主机画像 + 预检 + worst-of-N 校准 + 松弛系数）的体系。
- 治理层 (#806)：昂贵资源的触发权限用显式白名单管控。

一句话收束：今天没有惊天动地的新模型，但每一个 PR 都在回答同一个问题——“如何让一个多模态推理服务，既能加新能力，又能被可信地测量和守护？”这正是把研究原型变成生产系统的日常功夫。

建议的学习路线（从这里继续深挖）

1. 推理服务基础：continuous batching、KV cache、radix attention、CUDA graph、tensor parallelism (TP)——这些是 SGLang 的核心，本期 PR 多次擦边 (TP2、prefill batching、stage cache)。
2. 多模态/Omni 流水线：thinker → talker → code2wav 的分阶段生成，speaker embedding / vocoder / mel 频谱。
3. 异步并发：Python asyncio / aiohttp，Little's Law（并发~吞吐×延迟）。
4. 测试工程：性能回归的抖动治理、worst-of-N 校准、flaky test、GitHub Actions 编排。